ImagePipeline con coda Rigtorp nei communicator MoStream

Confronto Rigtorp vs Vyukov sul workload applicativo ImagePipeline — relazione tecnica, 27 ottobre 2026

> **Sintesi.** Il benchmark `Benchmarks/ImagePipeline/test_image_1.mojo` è stato compilato due volte, cambiando solo la coda usata dai communicator fra gli stadi (Vyukov, predefinita, oppure Rigtorp con `-D MOSTREAM_RIGTORP=1`), ed eseguito sulla macchina virtuale con Mojo 1.1.0. Con 5 ripetizioni per coda e due configurazioni di parallelismo, le mediane di throughput differiscono di **+0,25%** (10 thread) e **+0,35%** (34 thread) a favore di Rigtorp: differenze inferiori alla variabilità fra ripetizioni. **Su questo workload le due code sono equivalenti**; il costo dominante è il calcolo sulle immagini, non la comunicazione fra stadi.

1. Obiettivo

I benchmark sintetici (cartella `Benchmarks/BenchMark Vyukov vs RigTorp`) avevano indicato Rigtorp come una coda MPMC molto efficiente. L'obiettivo di questa attività era verificare se il vantaggio si mantiene in un'applicazione reale costruita con MoStream, usando Rigtorp come coda dei `Communicator` al posto di Vyukov, a parità di tutto il resto.

2. Come si seleziona la coda dei communicator

La selezione è già integrata nella libreria (vedi `docs/Rigtorp_integrazione_runtime.md`) e avviene **in compilazione**, nel file `MoStream/communication_queue.mojo`:

```
comptime USE_RIGTORP = is_defined["MOSTREAM_RIGTORP"]()
comptime COMMUNICATION_QUEUE_NAME = "rigtorp" if USE_RIGTORP else "vyukov"
```

`CommunicationQueue[T]` alloca soltanto la coda scelta e inoltra `push`, `try_push` e `try_pop` con un `comptime if`: non ci sono dispatch a runtime né code inutilizzate. `Communicator` (in `MoStream/communicator.mojo`) usa questa coda, quindi la scelta riguarda tutte le code fra gli stadi di `Pipeline.run()` e `Pipeline.run_cooperative()`. Le ready/wait queue dello scheduler cooperativo restano Vyukov.

- Senza definizione: Vyukov (predefinita).
- Con `-D MOSTREAM_RIGTORP=1`: Rigtorp. Conta solo la presenza della definizione, anche `=0` seleziona Rigtorp.
- All'avvio il runtime stampa `{MoStream} Inter-stage queue: rigtorp` (o `vyukov`): lo abbiamo usato per verificare ogni esecuzione.

Nessun file della libreria è stato modificato per questa attività: il codice sorgente non cambia, cambia solo il flag di compilazione.

3. Il workload ImagePipeline

`test_image_1.mojo` costruisce, con il runtime standard, la pipeline

```
TimedImageSource -> Grayscale -> GaussianBlur -> Sharpen -> ImageSink
```

- **TimedImageSource**: genera per 60 secondi copie di un'immagine a gradiente 512×512 (`W = H = 512`, `DURATION = 60`);
- **Grayscale**, **GaussianBlur** (3×3), **Sharpen** (3×3): filtri replicabili in parallelo;
- **ImageSink**: conta le immagini ricevute in un contatore atomico.

Il programma riceve da riga di comando i gradi di parallelismo dei cinque stadi, attiva il pinning (`pipeline.setPinning(True)`) e misura con `perf_counter_ns` il tempo di `pipeline.run()`. Il throughput stampato è *immagini arrivate al sink / tempo totale di run()*. Le code fra gli stadi hanno la capacità predefinita della Pipeline, **1024** elementi (potenza di due, valida per entrambe le code).

4. Preparazione dell'ambiente

4.1 Toolchain Mojo 1.1

Dal 26 ottobre la libreria richiede Mojo 1.1 (import `std.runtime._asyncrt`, atomici `Atomic[Int64]`, ecc.). Sul portatile il `mojo` di sistema era la 0.26.2 e non compilava la libreria; i compilatori in cache di pixi (1.0.0b*) erano privi della stdlib. Per una prima prova locale è stato quindi creato un ambiente pixi temporaneo con la stessa versione usata nei test di integrazione:

```
mkdir mojo11 && cd mojo11
pixi init -c https://conda.modular.com/max -c conda-forge .
pixi add "mojo==1.1.0"
pixi run mojo --version        # Mojo 1.1.0 (8189361e)
```

Il compilatore va invocato con `pixi run`: chiamando direttamente il binario dell'ambiente non viene trovata la stdlib (`error: unable to locate module 'std'`).

4.2 Adeguamento del benchmark a Mojo 1.1

Con la libreria aggiornata, la compilazione falliva nel benchmark stesso:

```
test_image_1.mojo:52:46: error: 'Atomic' parameter 'T' has 'Deinitable & Movable' type,
but value has type 'DType'
    var count_ptr = unsafe_alloc[Atomic[DType.int64]](1)
```

In Mojo 1.1 `Atomic` è parametrizzato sul tipo scalare e non su un `DType`. È la stessa migrazione già applicata alla libreria; è stata estesa a tutti i file di `Benchmarks/ImagePipeline` (`image_stages.mojo`, `test_image_1.mojo`, `test_image_1_coop.mojo`, `test_image_2.mojo`, `test_image_3.mojo`, `test_image_3_coop.mojo`):

```
cd Benchmarks/ImagePipeline
sed -i 's/Atomic\[DType\.int64\]/Atomic[Int64]/g' *.mojo
```

La sostituzione è puramente sintattica (stesso tipo a 64 bit con segno): non cambia né la logica dei filtri né la misura.

4.3 Libreria di pinning

Al primo avvio ogni worker falliva con

```
Raised: dlopen failed: ./MoStream/lib/libpinning.so: cannot open shared object file
```

perché l'helper C per il pinning non era stato compilato. Come indicato nel README:

```
make -C MoStream/lib            # produce libpinning.so e libpinning.o
export MOSTREAM_HOME="$PWD"     # radice del repository
```

5. Prova locale (smoke test)

```
cd mojo11
pixi run mojo build -I "$REPO" -I "$REPO/Benchmarks/ImagePipeline" -D MOSTREAM_RIGTORP=1
\
    "$REPO/Benchmarks/ImagePipeline/test_image_1.mojo" -o test_image_1_rigtorp
cd "$REPO" && export MOSTREAM_HOME="$PWD"
./test_image_1_rigtorp 1 2 4 2 1
```

```
  Configuration: Source=1 GrayScale=2 GaussianBlur=4 Sharpen=2 Sink=1
{MoStream} Standard MoStream runtime is used with 10 threads
{MoStream} CPU pinning is enabled
{MoStream} Inter-stage queue: rigtorp
{MoStream} ...terminated successfully!
Elapsed time: 61876.81991 ms
Throughput: 1349.5037418124482 img/s
```

La prova conferma che la pipeline con Rigtorp compila, gira e termina correttamente (EOS propagato fino al sink). Il portatile (12 thread hardware) non è una macchina da benchmark, quindi questo valore **non** va confrontato con quelli della sezione 7.

6. Campagna sulla macchina virtuale

6.1 Macchina

Voce	Valore
Host	ungolo@131.114.2.95 (hostname `xilinx`)
CPU	Intel Xeon Gold 5512U, 1 socket, 28 core / 56 thread, 1 nodo NUMA, max 2,1 GHz
Memoria	circa 251 GiB
Kernel	Linux 5.15.0-168-generic
Compilatore	Mojo 1.1.0 (8189361e), `/usr/local/bin/mojo`
Carico all'avvio	load average 0,00 / 0,00 / 0,00

6.2 Copia dei sorgenti

La cartella `~/MoStream` presente sulla VM è una versione precedente (non contiene `communication_queue.mojo`). Per non alterarla, libreria e benchmark sono stati copiati in una cartella nuova, `~/mostream-rigtorp-bench`, e l'helper di pinning è stato ricompilato lì:

```
ssh ungolo@131.114.2.95 'mkdir -p ~/mostream-rigtorp-bench/Benchmarks'
tar czf - --exclude='*.so' --exclude='*.o' MoStream Benchmarks/ImagePipeline \
  | ssh ungolo@131.114.2.95 'cd ~/mostream-rigtorp-bench && tar xzf - && make -C MoStrea
m/lib'
```

6.3 Compilazione dei due binari

I due binari differiscono **solo** per la definizione `MOSTREAM_RIGTORP`:

```
cd ~/mostream-rigtorp-bench && mkdir -p bin
mojo build -O3 -I . -I Benchmarks/ImagePipeline \
    Benchmarks/ImagePipeline/test_image_1.mojo -o bin/img1_vyukov
mojo build -O3 -I . -I Benchmarks/ImagePipeline -D MOSTREAM_RIGTORP=1 \
    Benchmarks/ImagePipeline/test_image_1.mojo -o bin/img1_rigtorp
```

6.4 Tracciabilità

Gli hash SHA-256 dei sorgenti delle code sulla VM coincidono con quelli locali e con quelli registrati nei test di integrazione del 26 ottobre (`Tests/risultati_runtime_queue/20261026-mojo1.1.json`, 144 configurazioni superate): il codice misurato è esattamente quello validato.

File	SHA-256
MoStream/communication_queue.mojo	`abbb5541…b75f`
MoStream/MPMC_queue_rigtorp.mojo	`875a17a0…0e7c`
MoStream/MPMC_queue.mojo (Vyukov)	`a79a5a39…5d57`
bin/img1_vyukov	`2b0dc2f8…b10b`
bin/img1_rigtorp	`f281496c…1fe7`

6.5 Protocollo di misura

- Due configurazioni (Source-Grayscale-Blur-Sharpen-Sink): **1-2-4-2-1** (10 thread) e **1-8-16-8-1** (34 thread, entro i 56 thread hardware e senza sovrascrittura);
- 5 ripetizioni per coda e configurazione, 20 esecuzioni da circa 60 s (circa 21 minuti in totale);
- ordine **alternato**: nelle ripetizioni dispari parte prima Vyukov, nelle pari Rigtorp, per non favorire sistematicamente una delle due con effetti di riscaldamento o termici;
- ogni esecuzione controlla che il runtime abbia stampato la coda attesa (`Inter-stage queue: …`), con un timeout di 300 s.

Script `~/mostream-rigtorp-bench/run.sh`, lanciato con `nohup ./run.sh > run.log 2>&1 &`:

```
#!/bin/bash
export MOSTREAM_HOME="$HOME/mostream-rigtorp-bench"
cd "$MOSTREAM_HOME"
OUT=results.csv
echo "config,rep,backend,elapsed_ms,throughput" > $OUT
for cfg in "1 2 4 2 1" "1 8 16 8 1"; do
  for rep in 1 2 3 4 5; do
    if (( rep % 2 )); then order="vyukov rigtorp"; else order="rigtorp vyukov"; fi
    for b in $order; do
      log=$(timeout 300 ./bin/img1_$b $cfg 2>&1)
      echo "$log" | grep -q "Inter-stage queue: $b" || echo "WARN backend mismatch $b" >
&2
      ms=$(echo "$log" | sed -n "s/Elapsed time: \(.*\) ms/\1/p")
      tp=$(echo "$log" | sed -n "s/Throughput: \(.*\) img\/s/\1/p")
      echo "${cfg// /-},$rep,$b,$ms,$tp" | tee -a $OUT
    done
  done
done
echo DONE
```

Tutte le 20 esecuzioni sono terminate correttamente, senza timeout né segnalazioni `WARN`, e il log termina con `DONE`.

7. Risultati

7.1 Statistiche per configurazione (throughput, img/s)

Config.	Coda	n	Mediana	Media	Min	Max	Dev. std (CV)
1-2-4-2-1	Vyukov	5	**10.161**	10.032	9.481	10.202	309 (3,08%)
1-2-4-2-1	Rigtorp	5	**10.186**	10.201	10.152	10.256	41 (0,41%)
Rigtorp rispetto a Vyukov (mediane)							**+0,25%**
1-8-16-8-1	Vyukov	5	**19.696**	19.736	19.619	19.957	130 (0,66%)
1-8-16-8-1	Rigtorp	5	**19.765**	19.708	19.557	19.793	99 (0,50%)
Rigtorp rispetto a Vyukov (mediane)							**+0,35%**

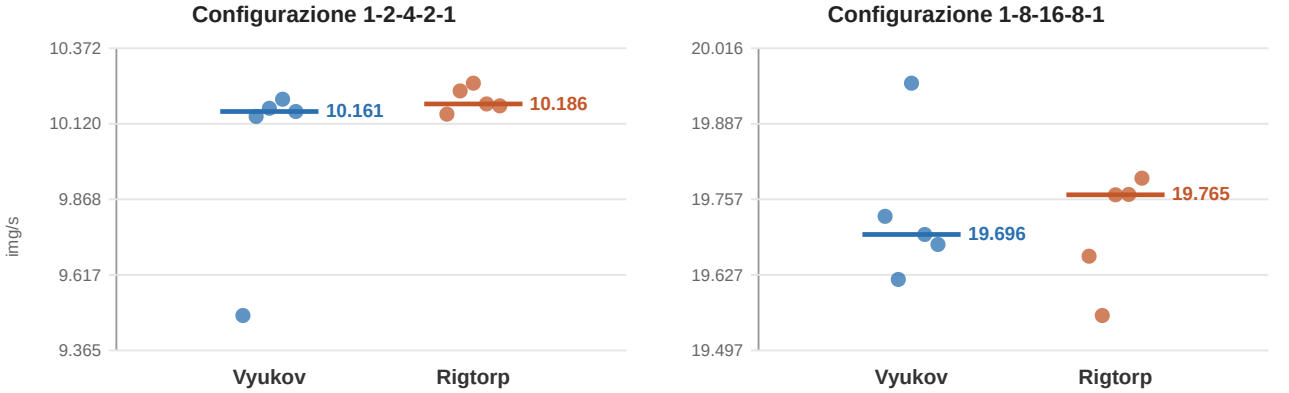


Figura 1 — Throughput delle singole ripetizioni (punti, spostati orizzontalmente in ordine di ripetizione) e mediana (segmento con valore). Le scale verticali dei due pannelli sono indipendenti e non partono da zero: l'intervallo mostrato è di poche centinaia di img/s, cioè pochi punti percentuali.

7.2 Confronto per coppie di esecuzioni consecutive

Config.	Rip.	Prima in ordine	Vyukov (img/s)	Rigtorp (img/s)	Rigtorp vs Vyukov
1-2-4-2-1	1	Vyukov	9.481	10.152	+7,08%
1-2-4-2-1	2	Rigtorp	10.145	10.230	+0,84%
1-2-4-2-1	3	Vyukov	10.172	10.256	+0,82%
1-2-4-2-1	4	Rigtorp	10.202	10.186	-0,15%
1-2-4-2-1	5	Vyukov	10.161	10.180	+0,19%
1-8-16-8-1	1	Vyukov	19.728	19.659	-0,35%
1-8-16-8-1	2	Rigtorp	19.619	19.557	-0,32%
1-8-16-8-1	3	Vyukov	19.957	19.765	-0,96%
1-8-16-8-1	4	Rigtorp	19.696	19.765	+0,35%
1-8-16-8-1	5	Vyukov	19.679	19.793	+0,58%

Rigtorp è avanti in 6 coppie su 10 (4 su 5 con 10 thread, 2 su 5 con 34 thread) e Vyukov nelle altre 4; nessuna differenza di coppia supera l’1% tranne la prima in assoluto (vedi 8.2).

8. Analisi

8.1 Le due code sono equivalenti su questo workload

Le differenze fra le mediane (+0,25% e +0,35%) sono dello stesso ordine della dispersione fra ripetizioni della stessa coda (deviazioni standard fra 0,4% e 3,1% della media). Con 5 ripetizioni non è possibile distinguerle dal rumore. Il risultato è coerente con la natura del workload: ogni messaggio è un'immagine 512×512 su cui vengono applicati filtri 3×3, cioè centinaia di migliaia di operazioni per messaggio, mentre un'operazione sulla coda costa poche decine di nanosecondi. Anche con 34 thread il tempo speso nelle code è una frazione trascurabile, quindi il vantaggio di Rigtorp osservato nei benchmark sintetici, dove le code sono il collo di bottiglia, qui non emerge. In compenso Rigtorp **non introduce regressioni**: si può usare come coda dei communicator senza perdere prestazioni.

8.2 La prima esecuzione

Nella configurazione 1-2-4-2-1 la prima ripetizione Vyukov (9 481 img/s) è circa il 7% più lenta delle altre ed è la causa quasi esclusiva della deviazione standard più alta di Vyukov (309 contro 41). Si tratta della primissima esecuzione della campagna, sulla macchina appena uscita dall'inattività: è plausibile un effetto di riscaldamento (frequenza della CPU, cache, page cache dei binari) e non una proprietà della coda. Escludendo quella esecuzione, le mediane di Vyukov e Rigtorp in 1-2-4-2-1 sono 10 167 e 10 186 img/s (+0,2%).

8.3 Scalabilità e tempo di run

Passando da 10 a 34 thread il throughput sale da circa 10 200 a circa 19 700 img/s (circa 1,9×) per entrambe le code: la scalabilità è limitata dal workload e dagli stadi sorgente e

sink, che restano a un solo thread, e non dalla scelta della coda. Il tempo totale di `run()` è circa 60,36 s in 1-2-4-2-1 e 60,00 s in 1-8-16-8-1. La sorgente si ferma a 60 s; l'eccedenza è il tempo per smaltire le immagini ancora in volo. Un'ipotesi da verificare è che con 10 thread gli stadi a valle siano più lenti della sorgente e le code (capacità 1024) siano piene alla chiusura.

9. Limiti

- **Campione piccolo:** 5 ripetizioni per caso, nessuna esecuzione di riscaldamento scartata.
- **Correttezza dei dati non verificata da questo test:** il sink conta le immagini, ma non verifica che ciascuna sia arrivata una sola volta né il contenuto. La correttezza delle code nei communicator è coperta dai test di integrazione (`Tests/run_runtime_queue_tests.py`, 144 configurazioni), non da questa campagna.
- **Ambito:** solo `test_image_1` (runtime standard), capacità delle code 1024, due configurazioni. Non sono stati misurati il runtime cooperativo (`test_image_1_coop`, `test_image_3_coop`) né le altre varianti.
- **Metrica:** il throughput include l'intero `run()` (avvio dei thread e smaltimento finale), non solo la fase a regime.

10. Possibili sviluppi

- Ripetere con più ripetizioni (ad esempio 20–50) e un'esecuzione di riscaldamento scartata, riportando gli intervalli di confidenza delle mediane.
- Ridurre il costo per messaggio (immagini più piccole) o la capacità delle code, per rendere la comunicazione una parte più rilevante del tempo e vedere se il divario dei benchmark sintetici riemerge.
- Estendere il confronto al runtime cooperativo e alle altre pipeline di `Benchmarks/ImagePipeline`.

Appendice A — Dati grezzi

File `~/mostream-rigtorp-bench/results.csv` sulla VM (copia locale salvata insieme a questa relazione).

Config.	Rip.	Coda	Tempo (ms)	Throughput (img/s)
1-2-4-2-1	1	Vyukov	60.356,1	9.481,1
1-2-4-2-1	1	Rigtorp	60.357,3	10.152,3
1-2-4-2-1	2	Rigtorp	60.354,4	10.230,2
1-2-4-2-1	2	Vyukov	60.358,2	10.145,1
1-2-4-2-1	3	Vyukov	60.357,1	10.172,0
1-2-4-2-1	3	Rigtorp	60.355,2	10.255,9
1-2-4-2-1	4	Rigtorp	60.356,7	10.186,4
1-2-4-2-1	4	Vyukov	60.357,5	10.202,0
1-2-4-2-1	5	Vyukov	60.355,9	10.161,2
1-2-4-2-1	5	Rigtorp	60.357,9	10.180,4
1-8-16-8-1	1	Vyukov	60.003,2	19.727,7
1-8-16-8-1	1	Rigtorp	60.001,9	19.659,1
1-8-16-8-1	2	Rigtorp	60.001,5	19.557,2
1-8-16-8-1	2	Vyukov	60.001,9	19.619,2
1-8-16-8-1	3	Vyukov	60.001,9	19.956,6
1-8-16-8-1	3	Rigtorp	60.002,6	19.764,8
1-8-16-8-1	4	Rigtorp	60.001,8	19.765,5
1-8-16-8-1	4	Vyukov	60.003,1	19.696,4
1-8-16-8-1	5	Vyukov	60.002,7	19.679,4
1-8-16-8-1	5	Rigtorp	60.001,7	19.793,3

Appendice B — File coinvolti

Percorso	Ruolo
MoStream/communication_queue.mojo	selezione della coda in compilazione (non modificato)
MoStream/communicator.mojo	communicator fra stadi, usa CommunicationQueue (non modificato)
MoStream/MPMC_queue_rigtorp.mojo, MPMC_queue.mojo	code Rigtorp e Vyukov (non modificate)
Benchmarks/ImagePipeline/*.mojo	adeguati a Mojo 1.1: `Atomic[DType.int64]` → `Atomic[Int64]`
MoStream/lib/libpinning.so	compilato con `make -C MoStream/lib`
VM: ~/mostream-rigtorp-bench/	copia di lavoro, binari `bin/`, `run.sh`, `run.log`, `results.csv`
docs/Rigtorp_integrazione_runtime.md	documentazione dell'integrazione e protocollo di confronto